

SmartBugs - Test Case Specification

Luciano Bercini

April 2026

Contents

1	Introduction	2
2	Relation with Pre-Modification Test Cases	2
3	Post-Modification Functional Requirements	2
4	Test Design Reference	3
5	Post-Modification Test Case Specification	4
5.1	Test Case TC-POST-01	4
5.2	Test Case TC-POST-02	4
5.3	Test Case TC-POST-03	4
5.4	Test Case TC-POST-04	5
5.5	Test Case TC-POST-05	5
5.6	Test Case TC-POST-06	5
5.7	Test Case TC-POST-07	5
5.8	Test Case TC-POST-08	6
5.9	Test Case TC-POST-09	6
5.10	Test Case TC-POST-10	6
6	Unchanged Regression Test Cases	6
7	Conclusion	7

1 Introduction

This document defines the post-modification test case specification for the evolved version of SmartBugs.

The proposed modification introduces an intelligent tool-selection mechanism based on feature extraction and machine learning models. The goal of the new workflow is to automatically select the most suitable SmartBugs analysis tools for a given smart contract, while preserving the behavior of the original manual workflow.

The purpose of this document is to specify the concrete test cases that will be used to validate the modified system. The testing strategy, pass/fail criteria, category partition, constraints, and derived test frames are defined in the separate *SmartBugs - Test Plan* document.

The test cases defined in this document are divided into:

- **unchanged test cases**, inherited from the pre-modification test specification and reused for regression testing;
- **added test cases**, introduced to validate the new intelligent workflow;
- **modified test cases**, if an existing test case requires adaptation;
- **removed test cases**, if a previous test case is no longer applicable.

2 Relation with Pre-Modification Test Cases

The pre-modification test case specification defined a set of functional test cases for the original SmartBugs workflow. These test cases covered the main original functionalities, including file collection, tool loading, task generation, Solidity compiler handling, and workflow orchestration.

Since the proposed modification is designed to be modular and backward compatible, the original test cases remain valid and are reused as regression tests.

Classification	Test Cases	Description
Unchanged	TC-PRE-01 – TC-PRE-11	Original SmartBugs workflow test cases reused for regression testing.
Added	TC-POST-01 – TC-POST-10	New test cases introduced to validate feature extraction, model inference, tool aggregation, intelligent workflow integration, and command-line configuration of the intelligent mode.
Modified	None	No existing pre-modification test case is modified at specification level.
Removed	None	No pre-modification test case is removed, since the original workflow must remain supported.

Table 1: Classification of post-modification test cases

3 Post-Modification Functional Requirements

The modified system introduces a new intelligent workflow while preserving the original SmartBugs behavior.

The following functional requirements are considered for post-modification testing.

Requirement	Description
FR_1: Feature Extraction	The system must be able to extract a structured set of features from a Solidity smart contract using the integrated feature extraction component.
FR_2: Feature Selection	The system must be able to select the subset of features required by the machine learning models.
FR_3: Model Inference	The system must be able to load and execute the pre-trained machine learning models.
FR_4: Tool Prediction Aggregation	The system must be able to aggregate model predictions and remove duplicate or irrelevant outputs.
FR_5: Intelligent Workflow Integration	The system must be able to execute the original SmartBugs pipeline using the tools selected by the intelligent workflow.
FR_6: Backward Compatibility	The system must preserve the original manual tool-selection workflow when the intelligent mode is not enabled.
FR_7: Intelligent CLI Configuration	The system must correctly parse and store the command-line options required to activate and configure the intelligent workflow.

Table 2: Post-modification functional requirements

4 Test Design Reference

The test cases specified in this document are derived from the category partition and the derived test frames defined in the *SmartBugs - Test Plan* document.

In particular, the added test cases TC-POST-01 – TC-POST-10 correspond to the representative test frames TF-POST-01 – TF-POST-10 defined in the test plan.

Test Case ID	Related Test Frame	Purpose
TC-POST-01	TF-POST-01	Validate successful feature extraction from a valid contract.
TC-POST-02	TF-POST-02	Validate feature extraction failure handling.
TC-POST-03	TF-POST-03	Validate selection of the required model features.
TC-POST-04	TF-POST-04	Validate prediction and selection of a single tool.
TC-POST-05	TF-POST-05	Validate prediction and selection of multiple tools.
TC-POST-06	TF-POST-06	Validate duplicate prediction removal.
TC-POST-07	TF-POST-07	Validate filtering of non-informative predictions.
TC-POST-08	TF-POST-08	Validate integration with the SmartBugs execution pipeline.
TC-POST-09	TF-POST-09	Validate backward compatibility with the manual workflow.
TC-POST-10	TF-POST-10	Validate parsing and storage of intelligent workflow command-line options.

Table 3: Mapping between post-modification test cases and test frames

5 Post-Modification Test Case Specification

5.1 Test Case TC-POST-01

Test Case ID	TC-POST-01
Related Test Frame	TF-POST-01
Requirement	FR_1
Scenario	Feature extraction from a valid Solidity contract.
Pre-conditions	The feature extraction component is available and a valid Solidity contract exists.
Input	A valid Solidity contract, such as a simple storage or ERC20 contract.
Expected Result	The system extracts a structured set of features from the input contract without errors.

Table 4: Test Case TC-POST-01

5.2 Test Case TC-POST-02

Test Case ID	TC-POST-02
Related Test Frame	TF-POST-02
Requirement	FR_1
Scenario	Feature extraction failure caused by an invalid Solidity contract.
Pre-conditions	The feature extraction component is available and an invalid Solidity file exists.
Input	A Solidity file containing syntax errors.
Expected Result	The system reports the extraction failure and does not continue with model inference.

Table 5: Test Case TC-POST-02

5.3 Test Case TC-POST-03

Test Case ID	TC-POST-03
Related Test Frame	TF-POST-03
Requirement	FR_2
Scenario	Selection of the feature subset required by the machine learning models.
Pre-conditions	A complete feature vector has been extracted from the contract.
Input	A feature vector containing all features produced by the feature extraction component.
Expected Result	The system selects the subset of features required by the trained models, including the 41 input features expected by the machine learning pipelines.

Table 6: Test Case TC-POST-03

5.4 Test Case TC-POST-04

Test Case ID	TC-POST-04
Related Test Frame	TF-POST-04
Requirement	FR_3, FR_4
Scenario	Prediction and selection of a single valid analysis tool.
Pre-conditions	The machine learning models are available and the input feature subset is valid.
Input	A valid feature subset passed to the model inference module.
Expected Result	The system predicts one valid SmartBugs tool and includes it in the final tool list.

Table 7: Test Case TC-POST-04

5.5 Test Case TC-POST-05

Test Case ID	TC-POST-05
Related Test Frame	TF-POST-05
Requirement	FR_3, FR_4
Scenario	Prediction and selection of multiple valid analysis tools.
Pre-conditions	The model inference module produces multiple valid tool predictions.
Input	A valid feature subset passed to multiple machine learning models.
Expected Result	The system aggregates the predictions and returns a final list containing multiple valid tools.

Table 8: Test Case TC-POST-05

5.6 Test Case TC-POST-06

Test Case ID	TC-POST-06
Related Test Frame	TF-POST-06
Requirement	FR_4
Scenario	Removal of duplicate tool predictions.
Pre-conditions	The model inference module produces duplicated tool predictions.
Input	A list of predicted tools containing duplicates, such as <code>slither</code> , <code>slither</code> , and <code>mythril</code> .
Expected Result	The system removes duplicate entries and keeps each selected tool only once.

Table 9: Test Case TC-POST-06

5.7 Test Case TC-POST-07

Test Case ID	TC-POST-07
Related Test Frame	TF-POST-07
Requirement	FR_4
Scenario	Filtering of non-informative predictions.
Pre-conditions	The model inference module produces one or more <code>nessun_tool</code> predictions.
Input	A list of predictions containing only <code>nessun_tool</code> , or a mix of valid tools and <code>nessun_tool</code> .
Expected Result	The system removes <code>nessun_tool</code> from the final tool list. If no valid tools remain, no SmartBugs task is generated or fallback behavior is applied.

Table 10: Test Case TC-POST-07

5.8 Test Case TC-POST-08

Test Case ID	TC-POST-08
Related Test Frame	TF-POST-08
Requirement	FR_5
Scenario	Execution of SmartBugs using automatically selected tools.
Pre-conditions	The intelligent workflow has produced a non-empty list of valid SmartBugs tools.
Input	A valid contract analyzed in intelligent mode.
Expected Result	The intelligent workflow is executed before the standard SmartBugs pipeline. The selected tools are stored in the runtime configuration, passed to the original SmartBugs workflow, tasks are generated, and the analysis pipeline is invoked.

Table 11: Test Case TC-POST-08

5.9 Test Case TC-POST-09

Test Case ID	TC-POST-09
Related Test Frame	TF-POST-09
Requirement	FR_6
Scenario	Preservation of the original manual workflow.
Pre-conditions	The modified system is installed and the intelligent mode is not enabled.
Input	A standard SmartBugs execution using manually specified tools.
Expected Result	When intelligent mode is not enabled through configuration or command-line options, the system behaves as the original SmartBugs workflow, using the manually specified tools without invoking the intelligent selection mechanism.

Table 12: Test Case TC-POST-09

5.10 Test Case TC-POST-10

Test Case ID	TC-POST-10
Related Test Frame	TF-POST-10
Requirement	FR_7
Scenario	Parsing and storage of intelligent workflow command-line options.
Pre-conditions	The modified SmartBugs command-line interface is available.
Input	A command line including options such as <code>--smart-select</code> , <code>--feature-extractor</code> , <code>--model-paths</code> , <code>-f</code> , and <code>--continue-on-errors</code> .
Expected Result	The command-line arguments are parsed correctly and the corresponding values are stored in the <code>Settings</code> object used by the intelligent workflow.

Table 13: Test Case TC-POST-10

6 Unchanged Regression Test Cases

The following pre-modification test cases are preserved and will be executed again after the modification as regression tests.

Test Case ID	Description	Classification
TC-PRE-01	Collection of a single valid Solidity file	Unchanged
TC-PRE-02	Collection of multiple Solidity files using a glob pattern	Unchanged
TC-PRE-03	Collection and task generation for a bytecode file	Unchanged
TC-PRE-04	Task generation for runtime bytecode	Unchanged
TC-PRE-05	Basic task generation for a Solidity file and a compatible tool	Unchanged
TC-PRE-06	Task generation for one Solidity file and multiple compatible tools	Unchanged
TC-PRE-07	Missing pragma with a solc-dependent tool	Unchanged
TC-PRE-08	Incompatible file and tool mode	Unchanged
TC-PRE-09	Loading a tool alias	Unchanged
TC-PRE-10	Handling result directory collisions	Unchanged
TC-PRE-11	Main SmartBugs orchestration workflow	Unchanged

Table 14: Unchanged test cases reused for regression testing

7 Conclusion

This document defines the post-modification test case specification for the evolved SmartBugs system.

The original pre-modification test cases are preserved and reused as regression tests, while new test cases are introduced to validate the added intelligent tool-selection workflow.

The post-modification test suite therefore verifies both the correctness of the new functionality and the preservation of the original SmartBugs behavior.